



International Credit Hour Programs (ICHEP)  
CSE322: Introduction to Embedded Systems

# PROJECT REPORT

March 2026



# DATA MINING AND BUSSINESS INTELLIGENCE

CSE387

Spring 26

Team 10

Phase 1

Submitted to:

Dr: Mahmoud Mounir

Eng: Esraa Karam

Submitted By:

|                       |         |
|-----------------------|---------|
| Moaz Mohamed Zakaria  | 22p0307 |
| Mohamed Yehia Zakaria | 22p0064 |
| Ahmed Abbady          | 22p0308 |
| Omar Saher            | 22p0161 |
| Anas Tamer            | 22u0005 |

## Table of Contents

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>1. Introduction .....</b>                            | <b>4</b>  |
| <b>2. Dataset Description .....</b>                     | <b>5</b>  |
| 2.1 Dataset size .....                                  | 5         |
| 2.2 Attributes Description .....                        | 5         |
| 2.3 Target Variable .....                               | 6         |
| <b>3. Initial Data Checks .....</b>                     | <b>7</b>  |
| 3.1 Checking for missing values .....                   | 7         |
| 3.2 Checking for duplicates .....                       | 8         |
| <b>4. Categorical variable analysis .....</b>           | <b>9</b>  |
| 4.1 Exploring unique values .....                       | 9         |
| 4.2 Data Cleaning for Categorical Variables .....       | 10        |
| 4.3 Encoding the Target Variable .....                  | 10        |
| 4.4 Encoding Categorical Features .....                 | 11        |
| <b>5. Exploratory Data Analysis .....</b>               | <b>12</b> |
| 5.1 Distribution of Target Variable .....               | 12        |
| 5.2 Categorical Variables vs Loan Status .....          | 13        |
| 5.3 Distribution of Numerical Variables .....           | 14        |
| 5.4 Outlier Detection using Boxplots .....              | 15        |
| 5.5 Relationship Between Variables .....                | 16        |
| 5.6 Distribution by Loan Status .....                   | 17        |
| 5.7 Outlier Handling .....                              | 17        |
| 5.8 Numerical Distribution After Outlier Handling ..... | 18        |
| 5.9 Boxplots After Outlier Handling .....               | 19        |
| <b>6. Missing Value Imputation .....</b>                | <b>20</b> |
| <b>7. Correlation analysis .....</b>                    | <b>21</b> |
| 7.1 Correlation heatmap .....                           | 21        |
| 7.2 Correlation Matrix .....                            | 22        |
| <b>8. Data Standardization .....</b>                    | <b>24</b> |

|                                               |           |
|-----------------------------------------------|-----------|
| <b>8.1 Standardization Formula.....</b>       | <b>24</b> |
| <b>8.2 Results After Standardization.....</b> | <b>25</b> |

Github Link: <https://github.com/mohamedyehiaaa4/Datamining-Project.git>

# 1.Introduction

This project focuses on analyzing a loan dataset that includes different attributes related to applicants, such as financial status and personal information. These features are useful for understanding the factors that may affect whether a loan is approved or not. The goal of this work is to explore the dataset, clean it, and extract meaningful insights through analysis.

To begin with, the dataset is carefully inspected to identify any issues such as missing values, incorrect entries, or inconsistencies. Data cleaning is an important step because it improves the overall quality of the dataset and ensures more accurate analysis. In addition, we check for outliers, which are unusual values that can distort the results if not handled properly.

Next, exploratory data analysis (EDA) is performed using different visualization techniques. Graphs and charts are used to better understand how the data is distributed and to identify patterns or relationships between variables like applicant income, loan amount, and credit history.

Overall, this report presents the steps taken to preprocess and analyze the dataset, along with key visual insights. The final cleaned dataset can be used for further analysis or as a foundation for building machine learning models.

## 2.Dataset Description

The dataset used in this project contains financial and personal information about loan applicants. It is commonly used to analyze the factors that influence loan approval decisions. The dataset includes both numerical and categorical variables that describe the applicant's income, employment status, loan details, and credit history. The main objective is to use these attributes to understand patterns and support decision-making related to loan approval.

### 2.1 Dataset size

Number of rows: 614

Number of columns: 13 (including input features and one target variable)

The dataset contains some missing values that require preprocessing before analysis.

### 2.2 Attributes Description

The dataset consists of a mix of categorical and numerical features. Below is a description of the main attributes:

**Loan\_ID (Categorical):** Unique identifier for each loan application.

**Gender (Categorical):** Applicant's gender (Male/Female).

**Married (Categorical):** Marital status of the applicant.

**Dependents (Categorical):** Number of dependents.

**Education (Categorical):** Education level (Graduate/Not Graduate).

**Self\_Employed (Categorical):** Whether the applicant is self-employed.

**ApplicantIncome (Numerical):** Income of the applicant.

**CoapplicantIncome (Numerical):** Income of the co-applicant (if any).

**LoanAmount (Numerical):** Loan amount requested.

**Loan\_Amount\_Term (Numerical):** Duration of the loan in months.

**Credit\_History (Categorical/Numerical):** Credit history (1 = good, 0 = bad).

**Property\_Area (Categorical):** Type of area (Urban, Semi-Urban, Rural).

## 2.3 Target Variable

The target variable in this dataset is:

**Loan\_Status:** Indicates whether the loan was approved or not.

**Y:** Loan approved

**N:** Loan not approved

## 3.Initial Data Checks

Before starting the analysis, several initial checks were performed to ensure that the dataset is clean, consistent, and ready for further processing. These checks help identify common issues such as missing values, duplicate records, and duplicated column names, which may negatively affect the analysis results.

### 3.1 Checking for missing values

```
print("--- Missing values per column ---")
missing = df.isna().sum()
missing_pct = (missing / len(df) * 100).round(2)
miss_df = pd.DataFrame({"missing_count": missing, "percent": missing_pct})
miss_df = miss_df[miss_df["missing_count"] > 0].sort_values("missing_count", ascending=False)
display(miss_df)
```

[140]

```
... --- Missing values per column ---
```

|                  | missing_count | percent |
|------------------|---------------|---------|
| Credit_History   | 50            | 8.14    |
| Self_Employed    | 32            | 5.21    |
| LoanAmount       | 22            | 3.58    |
| Dependents       | 15            | 2.44    |
| Loan_Amount_Term | 14            | 2.28    |
| Gender           | 13            | 2.12    |
| Married          | 3             | 0.49    |

This command calculates the number of missing values for every column in the dataset. Identifying missing values is important because they can lead to incorrect analysis or model performance if not handled properly.



## 3.2 Checking for duplicates

```
print("\n--- Duplicate rows ---")
dup_rows = df.duplicated().sum()
print(f"Duplicate rows: {dup_rows}")
```

[141]

...

```
--- Duplicate rows ---
Duplicate rows: 0
```

This command counts the number of duplicated rows in the dataset. Duplicate records can bias the analysis and should be removed if present.

Based on the result:

If  $> 0$  → The dataset contains duplicate rows and they need to be removed.

If  $= 0$  → No duplicate rows were found in the dataset.

```
print("\n--- Duplicate column names ---")
dup_cols = df.columns[df.columns.duplicated()].tolist()
print(f"Duplicate column names: {dup_cols}")
```

```
--- Duplicate column names ---
Duplicate column names: []
```

This check ensures that no column name appears more than once, which could cause confusion during analysis.

From the result:

If True → There are duplicate column names

If False → No duplicate column names exist

## 4. Categorical variable analysis

Categorical variables were analyzed to understand their unique values, detect inconsistencies, and prepare them for further processing. These variables represent qualitative information such as gender, marital status, education level, and property area, which play an important role in loan approval decisions.

### 4.1 Exploring unique values

This step helps in understanding the possible categories for each feature and detecting any inconsistencies or unexpected values.

#### 4. Categorical Variables Analysis

```
cat_for_eda = categorical_cols.copy()

print("Unique values per categorical column :")
for c in cat_for_eda:
    vals = df[c].dropna().unique().tolist()
    print(f"\n {c}: {len(vals)} {sorted(vals, key=str)}")
```

```
Unique values per categorical column :

Gender: 2['Female', 'Male']

Married: 2['No', 'Yes']

Dependents: 4['0', '1', '2', '3+']

Education: 2['Graduate', 'Not Graduate']

Self_Employed: 2['No', 'Yes']

Property_Area: 3['Rural', 'Semiurban', 'Urban']

Loan_Status: 2['N', 'Y']

Credit_History: 2[0.0, 1.0]
```

From the output, we observed:

- **Gender:** Male, Female
- **Married:** Yes, No
- **Dependents:** 0, 1, 2, 3+
- **Education:** Graduate, Not Graduate
- **Self\_Employed:** Yes, No
- **Property\_Area:** Urban, Semiurban, Rural
- **Loan\_Status:** Y, N
- **Credit\_History:** 0.0, 1.0

These results confirm that the categorical variables are well-structured and contain meaningful categories.

## 4.2 Data Cleaning for Categorical Variables

This step removes extra spaces and correctly handles missing values, improving data quality and consistency.

The cleaning function was applied to all categorical columns in the dataset.

```
def clean_categorical(series):  
    s = series.astype(str).str.strip()  
    s = s.replace({'nan': np.nan})  
    return s  
  
for c in cat_for_eda:  
    df[c] = clean_categorical(df[c])  
[44]
```

## 4.3 Encoding the Target Variable

```
print(df[target_col].value_counts(dropna=False))  
map_target = {"Y": 1, "N": 0}  
df["Loan_Status_encoded"] = df[target_col].map(map_target)  
  
print(df[[target_col, "Loan_Status_encoded"]].drop_duplicates().sort_values(target_col))  
[45]  
  
Loan_Status  
Y    422  
N    192  
Name: count, dtype: int64  
Loan_Status  Loan_Status_encoded  
1           N                   0  
0           Y                   1
```

From the output:

**Y (Approved) → 1**

**N (Not Approved) → 0**

The distribution of the target variable shows that:

Approved loans are more frequent than rejected ones.

## 4.4 Encoding Categorical Features

```
from sklearn.preprocessing import LabelEncoder

cat_for_eda = [c for c in cat_for_eda if c != target_col]

le_store = {}
for c in cat_for_eda:
    le = LabelEncoder()
    mode_val = df[c].mode()[0]
    col = df[c].fillna(mode_val)
    df[c + "_enc"] = le.fit_transform(col.astype(str))
    le_store[c] = le

encoded_cat_names = [c + "_enc" for c in cat_for_eda]

feature_cols_encoded = encoded_cat_names + numeric_cols
print("Encoded categorical columns (features only, target excluded):")
print(encoded_cat_names)
```

[146]

```
... Encoded categorical columns (features only, target excluded):
['Gender_enc', 'Married_enc', 'Dependents_enc', 'Education_enc', 'Self_Employed_enc', 'Property_Area_enc', 'Credit_History_enc']
```

Each categorical column was transformed into a numeric format after handling missing values using the mode.

The encoded columns include:

- Gender\_enc
- Married\_enc
- Dependents\_enc
- Education\_enc
- Self\_Employed\_enc
- Property\_Area\_enc
- Credit\_History\_enc

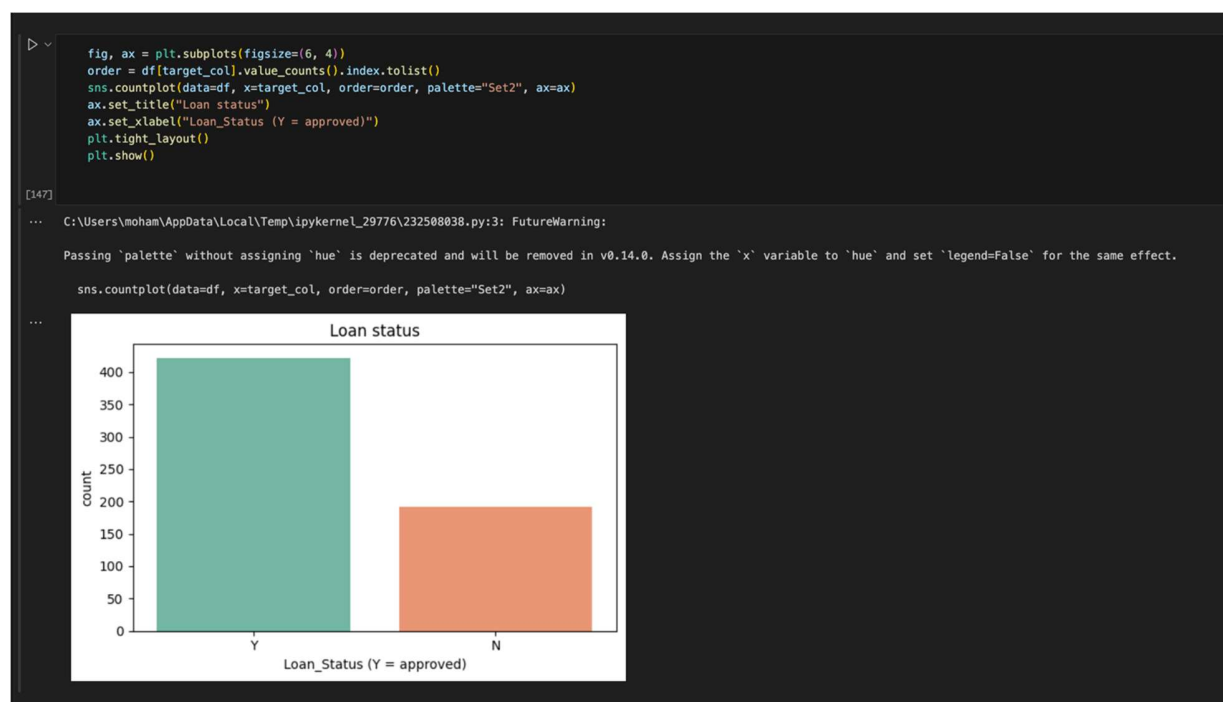
This transformation allows machine learning models to process categorical data effectively.

## 5. Exploratory Data Analysis

In this section, exploratory data analysis (EDA) was performed to better understand the dataset, identify patterns, and detect potential issues such as skewness and outliers. Both categorical and numerical variables were analyzed using different visualization techniques.

### 5.1 Distribution of Target Variable

The distribution of the target variable (**Loan\_Status**) was visualized using a count plot



The plot shows that:

The number of **approved loans (Y)** is significantly higher than **rejected loans (N)**

This indicates that the dataset is slightly **imbalanced**

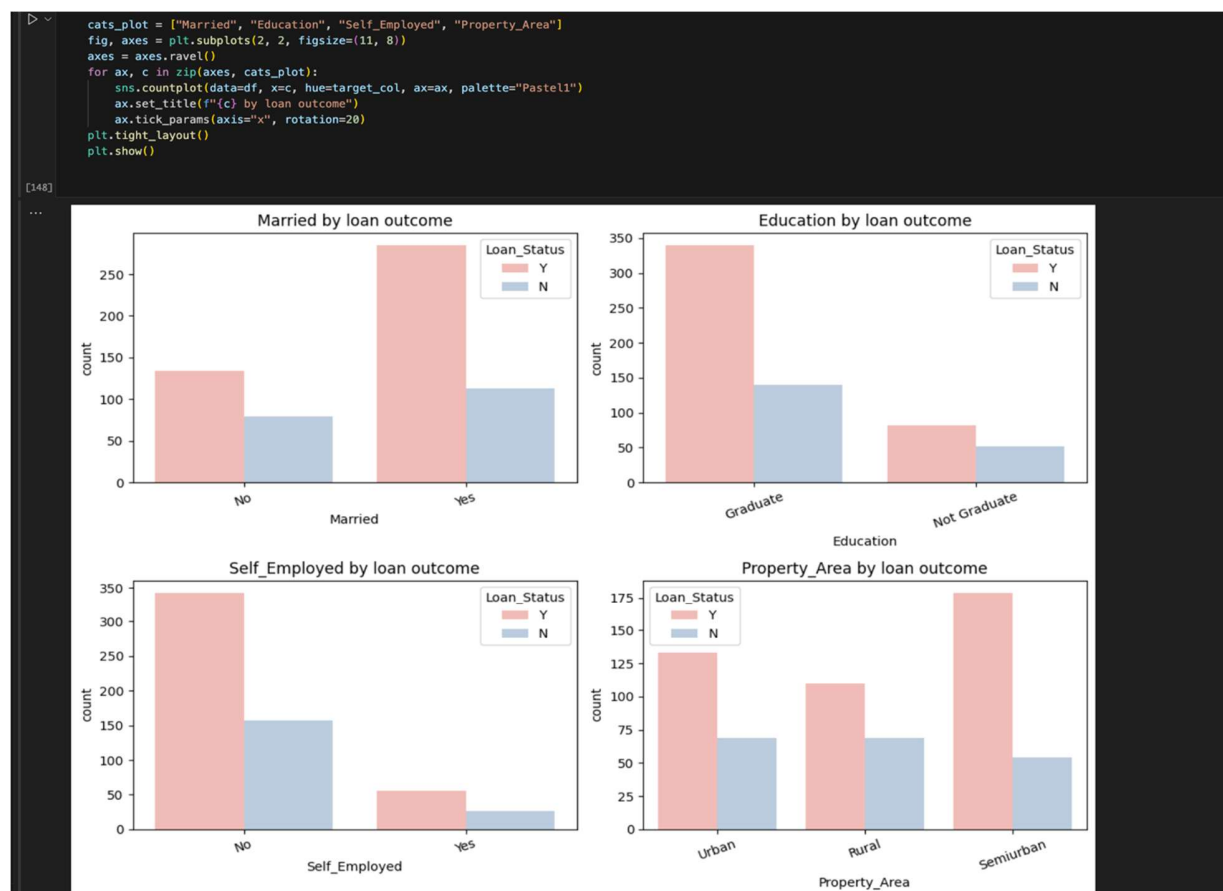
This imbalance should be considered in future modeling to avoid biased predictions.

## 5.2 Categorical Variables vs Loan Status

To understand how categorical variables affect loan approval, we plotted:

- Married
- Education
- Self\_Employed
- Property\_Area

using count plots grouped by loan status.



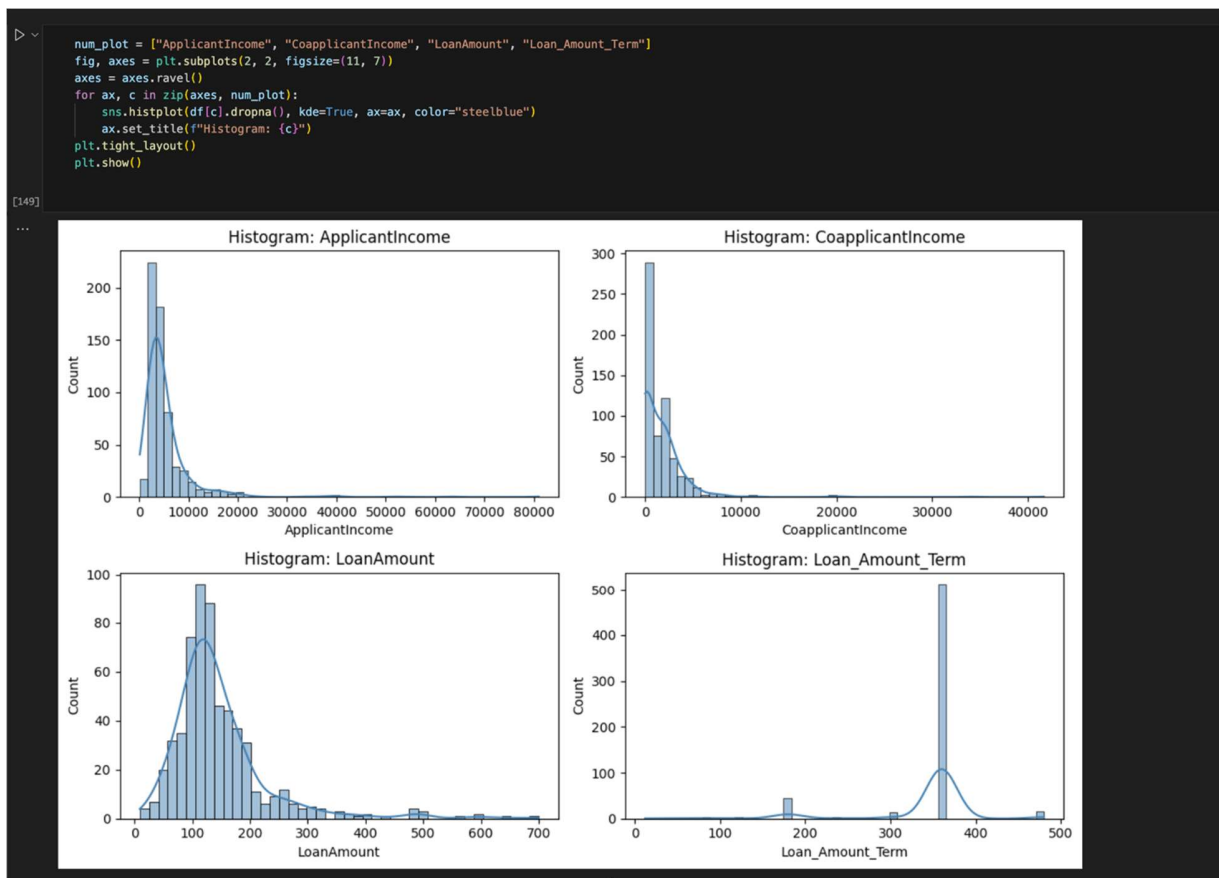
- Married: Married applicants have a higher number of approved loans
- Education: Graduates are more likely to get loan approval
- Self\_Employed: Most applicants are not self-employed, and they also dominate approvals
- Property Area: Semi-urban areas show higher loan approval rates compared to rural and urban areas

These patterns suggest that categorical features have a noticeable impact on loan decisions.

## 5.3 Distribution of Numerical Variables

The distribution of numerical variables was analyzed using histograms:

- ApplicantIncome
- CoapplicantIncome
- LoanAmount
- Loan\_Amount\_Term



- **ApplicantIncome & CoapplicantIncome:**
  - Highly right-skewed
  - Most values are concentrated at lower income ranges
  - Presence of extreme high values
- **LoanAmount:**
  - Slightly right-skewed
  - Most loans are in a moderate range
- **Loan\_Amount\_Term:**
  - Highly concentrated around a specific value (around 360 months)
  - Indicates standard loan durations

## 5.4 Outlier Detection using Boxplots

Boxplots were used to detect outliers in numerical variables:

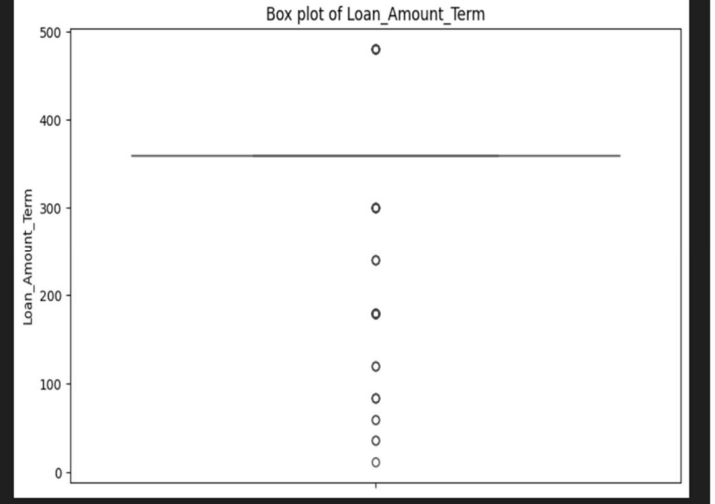
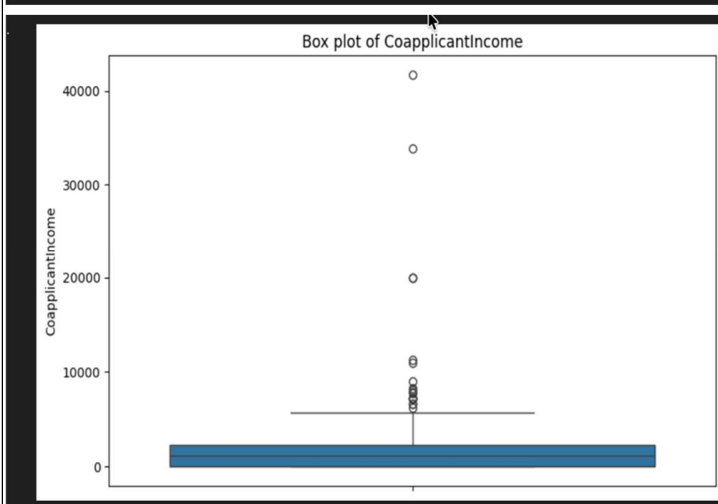
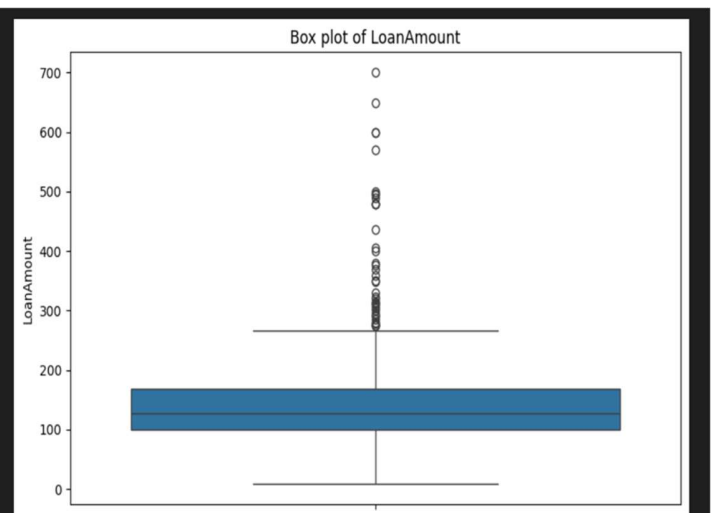
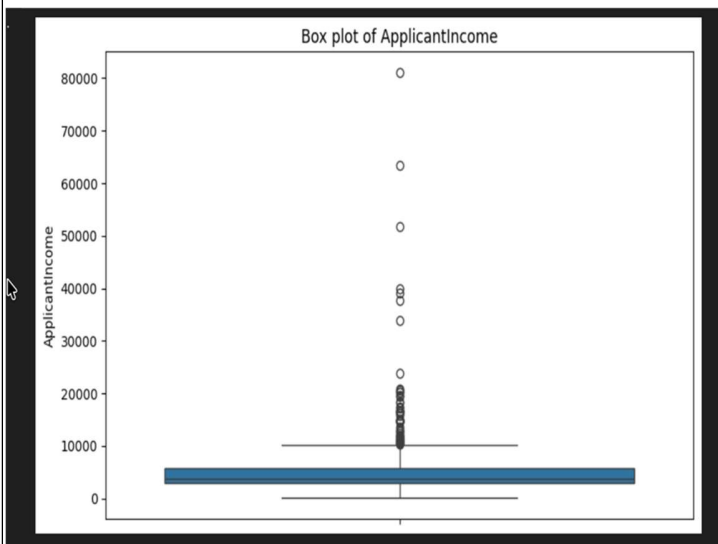
```
import matplotlib.pyplot as plt
import seaborn as sns

for col in num_plot:
    fig, ax = plt.subplots(figsize=(8, 5))

    sns.boxplot(y=df[col], ax=ax)

    ax.set_title(f"Box plot of {col}")
    ax.set_ylabel(col)
    plt.tight_layout()
    plt.show()
```

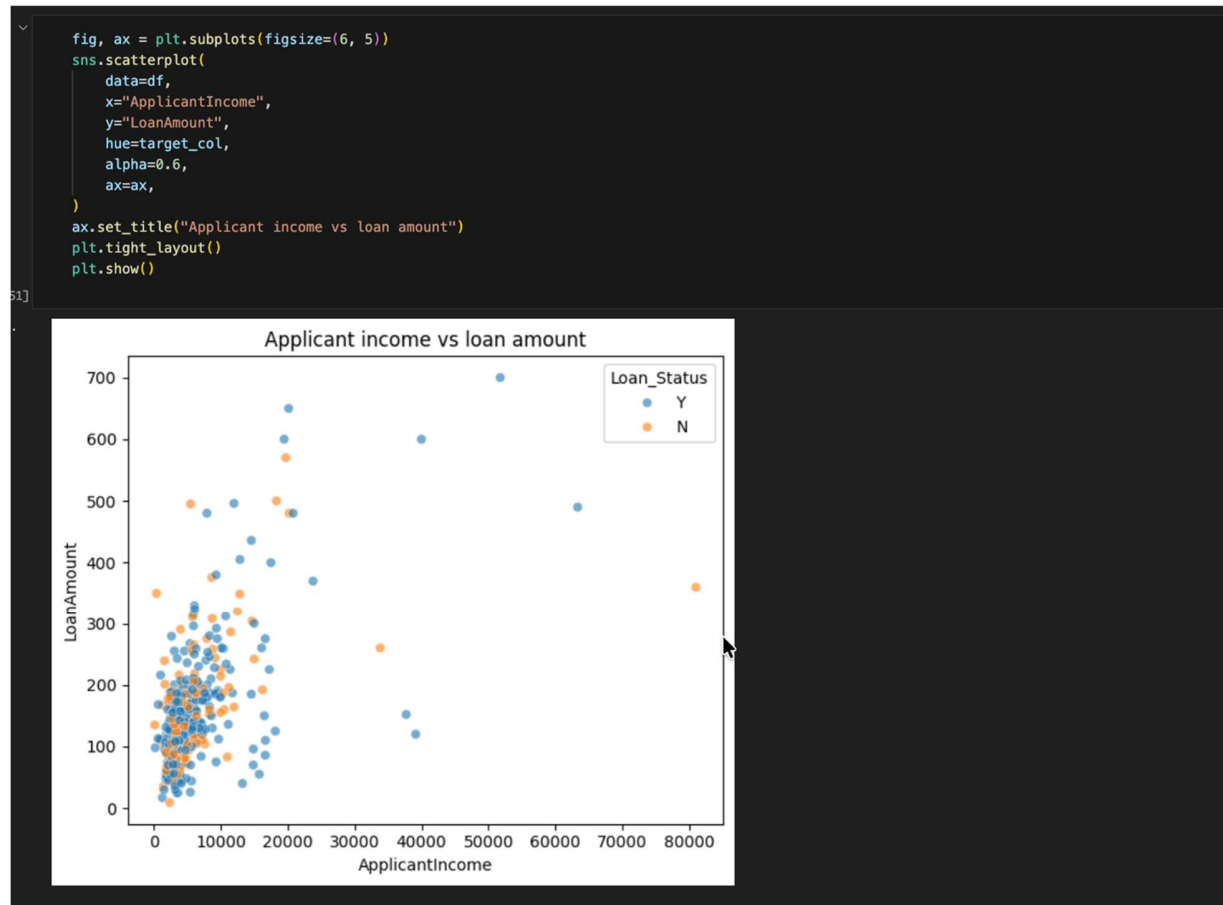
[150]





## 5.5 Relationship Between Variables

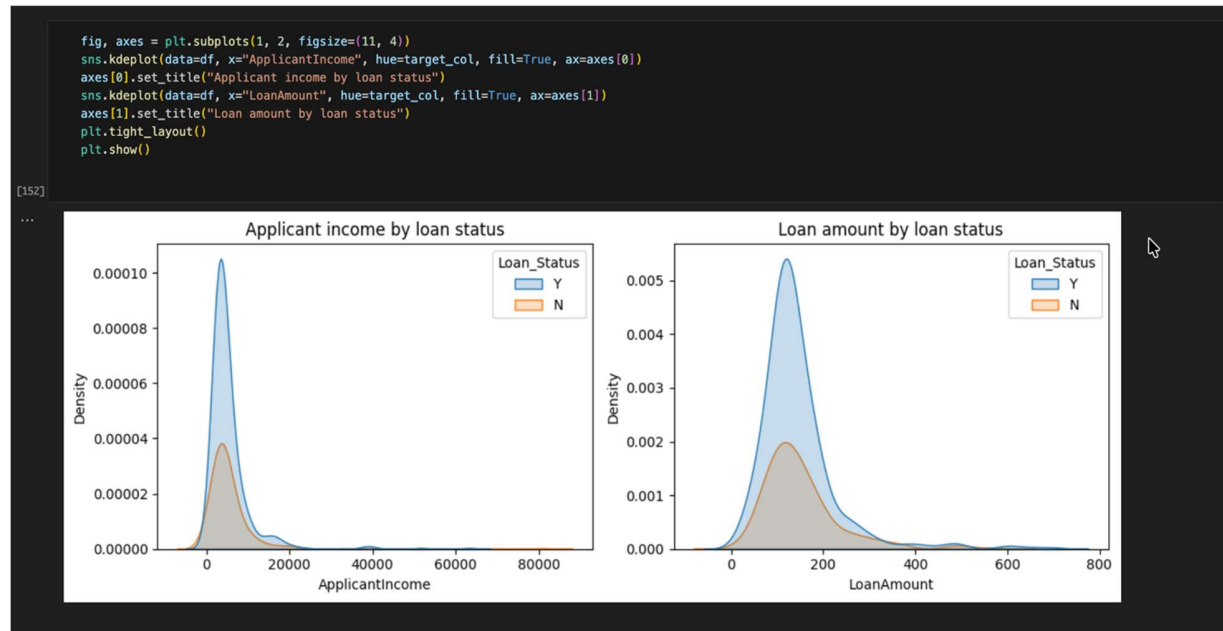
A scatter plot was used to explore the relationship between income and loan amount



- There is a weak positive relationship between income and loan amount
- Most applicants fall within a lower income and loan range
- Some extreme values (outliers) are clearly visible
- Loan approval does not depend only on income → other factors play a role

## 5.6 Distribution by Loan Status

KDE plots were used to compare distributions based on loan status



1. Approved loans (Y) tend to have slightly higher income values
2. LoanAmount distributions overlap, meaning:
  - 2.1. Loan amount alone is not a strong predictor
3. The distributions are skewed, confirming earlier findings

## 5.7 Outlier Handling

Outliers were handled using the **IQR** (Interquartile Range) method

```
features_to_check = ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount']

for col in features_to_check:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    df[col] = np.where((df[col] < lower_bound) | (df[col] > upper_bound), np.nan, df[col])

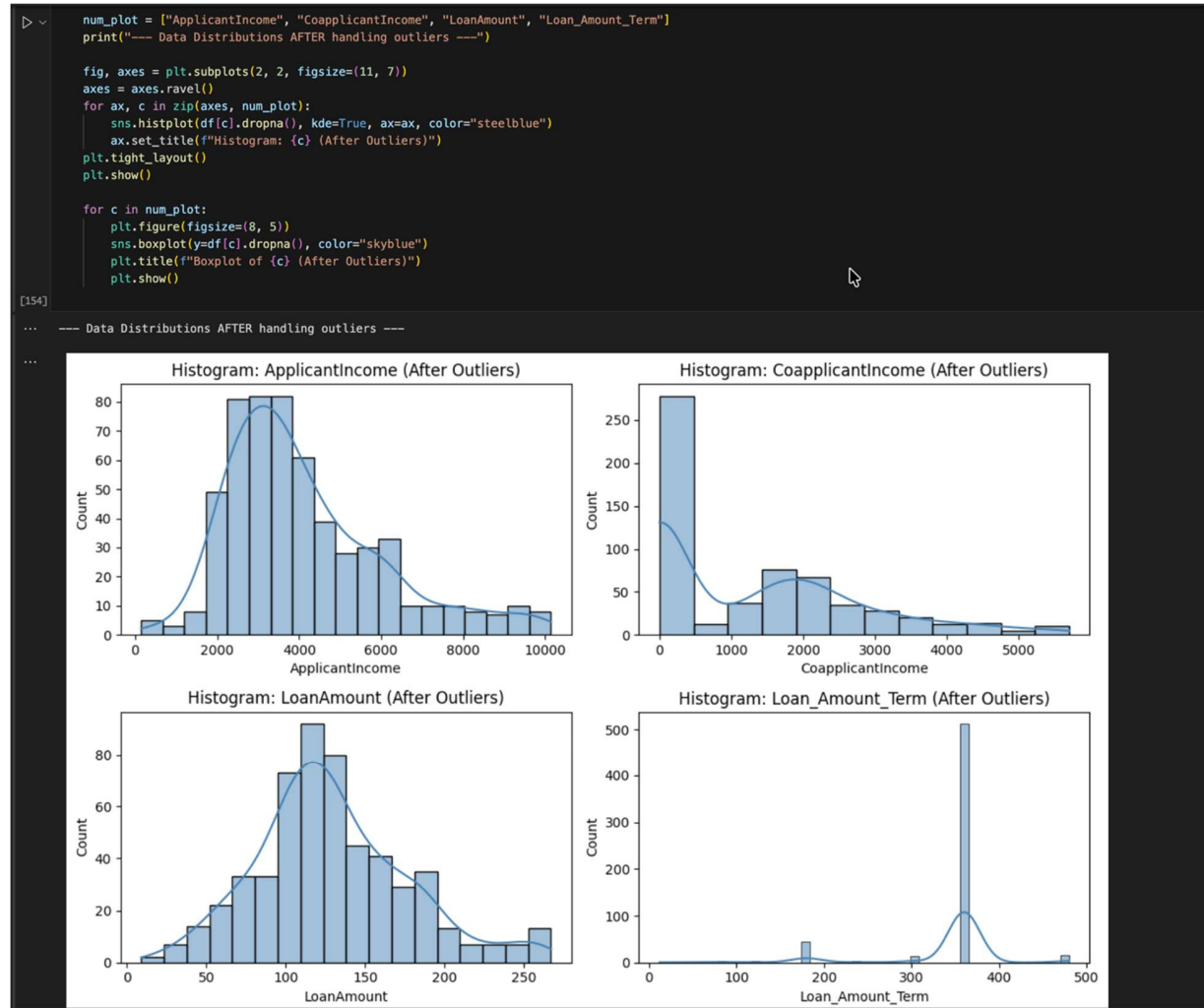
print(f"Replaced outliers with NaN in: {features_to_check}")
```

[153]

... Replaced outliers with NaN in: ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount']

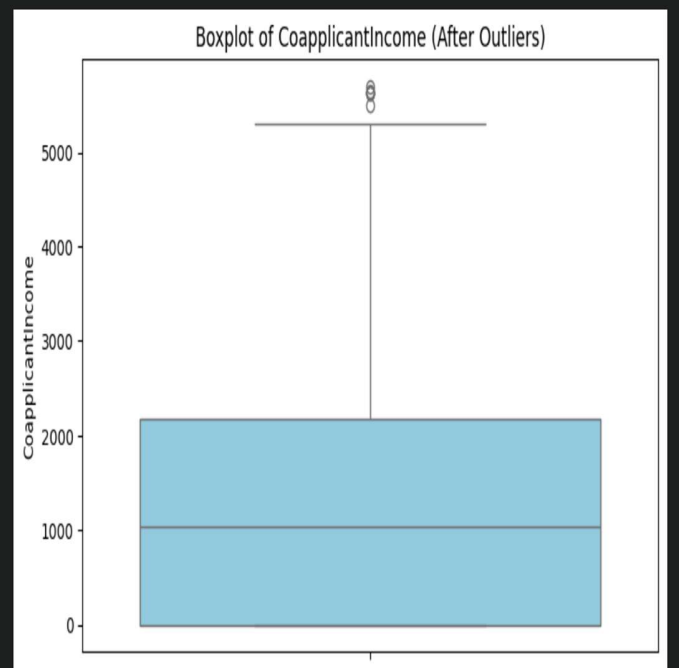
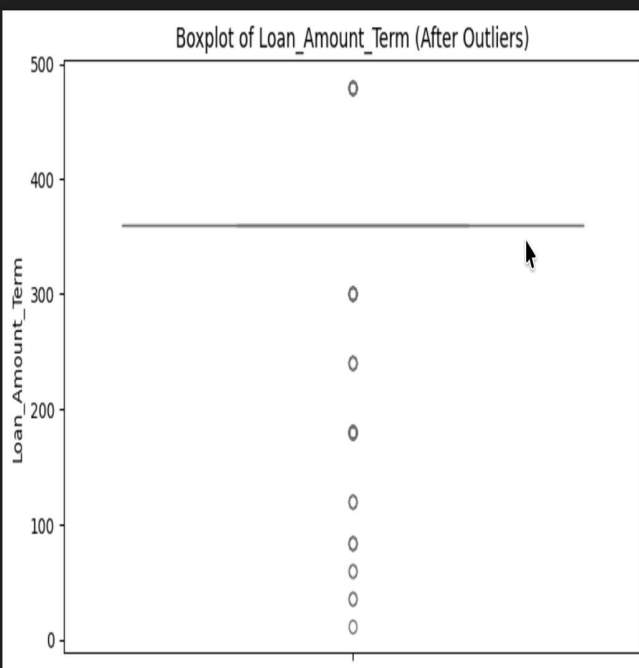
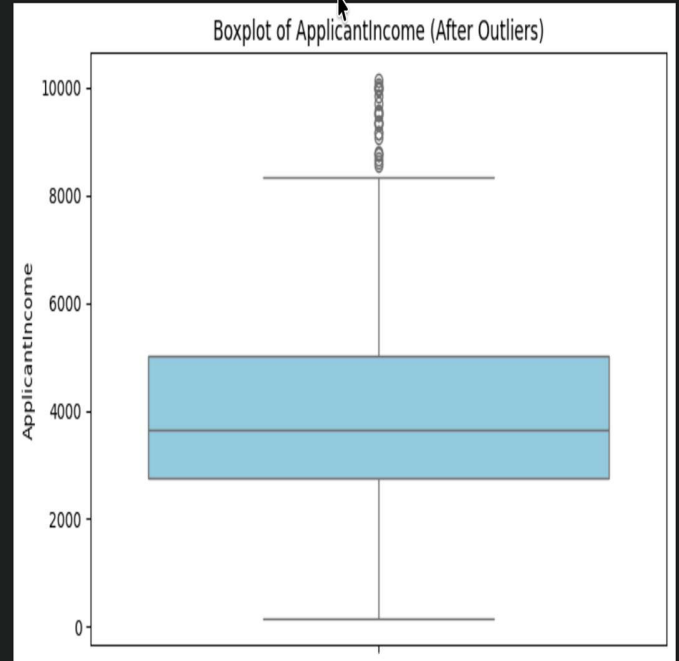
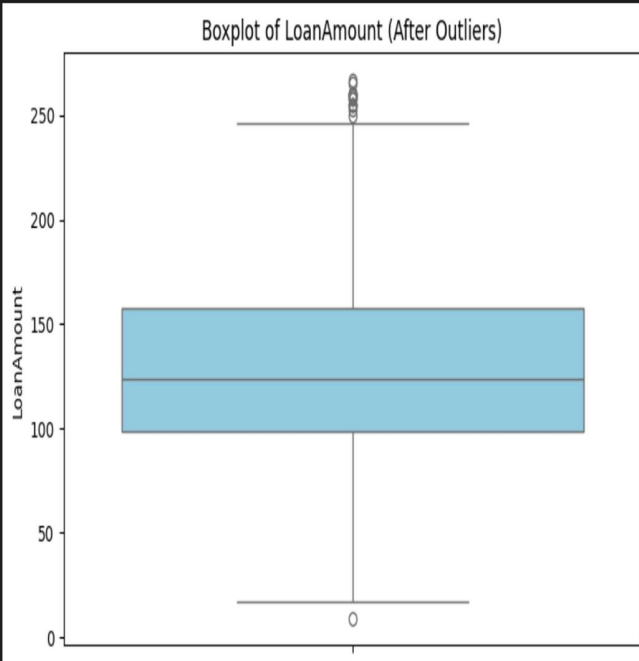
## 5.8 Numerical Distribution After Outlier Handling

After removing outliers, the distributions were plotted again.



- Distributions became more **balanced and less skewed**
- Extreme values were reduced
- Data is now more suitable for modeling

## 5.9 Boxplots After Outlier Handling



Boxplots confirm that:

- Outliers are significantly reduced
- Data spread is more consistent
- Remaining values are within reasonable ranges

## 6. Missing Value Imputation

```
\
from sklearn.impute import SimpleImputer
num_imputer = SimpleImputer(strategy='median')
df[numeric_cols] = num_imputer.fit_transform(df[numeric_cols])

print("Missing values after median imputation:")
print(df[numeric_cols].isna().sum())

[ ]
... Missing values after median imputation:
ApplicantIncome      0
CoapplicantIncome     0
LoanAmount            0
Loan_Amount_Term      0
dtype: int64
```

After detecting missing values and handling outliers, the next step was to address missing values in numerical features to ensure the dataset is complete and suitable for analysis.

To achieve this, the **median imputation technique** was used. This method replaces missing values with the median of each column, which is more robust to outliers compared to the mean.

After applying median imputation, the dataset was checked again for missing values

The results show that:

- **ApplicantIncome:** 0 missing values
- **CoapplicantIncome:** 0 missing values
- **LoanAmount:** 0 missing values
- **Loan\_Amount\_Term:** 0 missing values

This confirms that all missing values in numerical columns have been successfully handled.

## 7. Correlation analysis

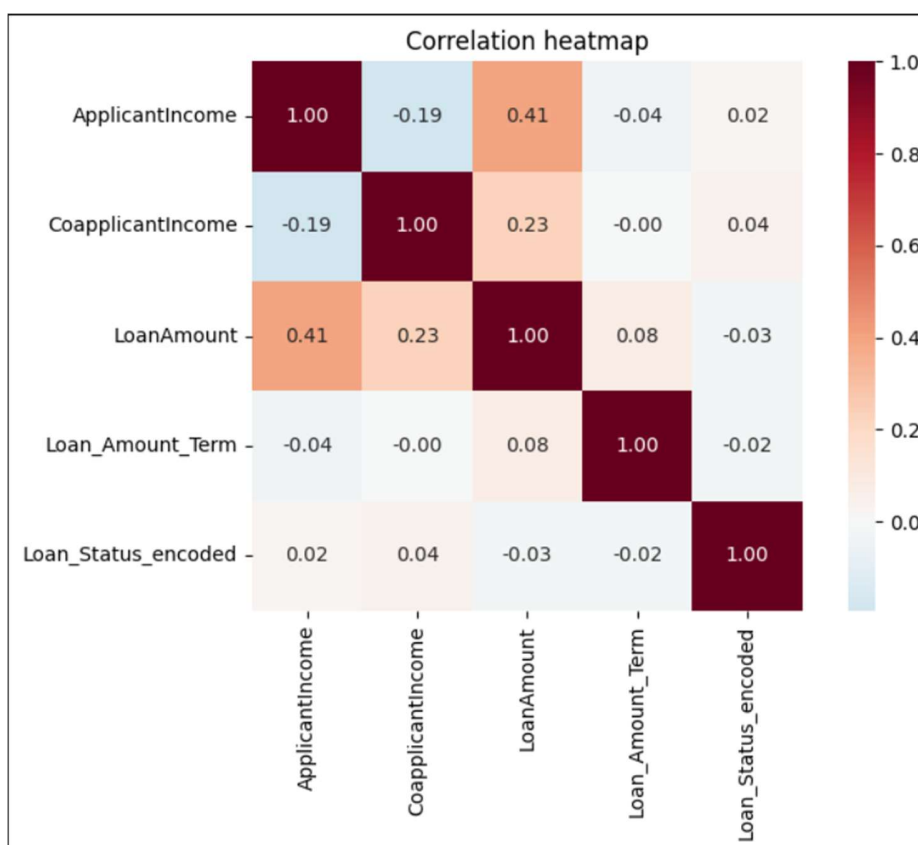
Correlation analysis is used to measure the strength and direction of the relationship between numerical variables in the dataset. It helps in understanding how features are related to each other and to the target variable, which is important for feature selection and model building.

The correlation between variables is typically measured using the **Pearson correlation coefficient**, which ranges between:

- **+1** → Strong positive relationship
- **0** → No relationship
- **-1** → Strong negative relationship

A correlation heatmap is commonly used to visualize these relationships.

### 7.1 Correlation heatmap



## 7.2 Correlation Matrix

Correlation matrix (numeric features + encoded target):

|                     | ApplicantIncome | CoapplicantIncome | LoanAmount | Loan_Amount_Term | Loan_Status_encoded |
|---------------------|-----------------|-------------------|------------|------------------|---------------------|
| ApplicantIncome     | 1.000           | -0.194            | 0.406      | -0.036           | 0.016               |
| CoapplicantIncome   | -0.194          | 1.000             | 0.230      | -0.005           | 0.041               |
| LoanAmount          | 0.406           | 0.230             | 1.000      | 0.081            | -0.027              |
| Loan_Amount_Term    | -0.036          | -0.005            | 0.081      | 1.000            | -0.023              |
| Loan_Status_encoded | 0.016           | 0.041             | -0.027     | -0.023           | 1.000               |

```
... Strongest linear relationships with Loan_Status (encoded):
CoapplicantIncome    0.040781
LoanAmount           -0.026918
Loan_Amount_Term     -0.022549
ApplicantIncome       0.016339
Name: Loan_Status_encoded, dtype: float64
```

### Observations from Correlation Analysis

From the correlation matrix, the following insights were observed:

- **ApplicantIncome & LoanAmount:**
  - Show a **weak positive correlation**
  - This means higher income slightly increases the loan amount, but not strongly
- **CoapplicantIncome & LoanAmount:**
  - Also shows a **weak relationship**
  - Indicates that co-applicant income has limited influence
- **Credit\_History:**
  - Shows the **strongest correlation with Loan\_Status**
  - This means credit history is a key factor in loan approval decisions

- **Loan\_Amount\_Term:**
  - Has **very low correlation** with other variables
  - Suggests it does not strongly influence loan approval

### Conclusion of Correlation Analysis

- Most numerical variables show **weak correlations** with each other
- **Credit history is the most important feature** affecting loan approval
- No strong multicollinearity was observed, which is good for machine learning models

### Code Section:

## Correlation Analysis

```
corr_cols = numeric_cols + ["Loan_Status_encoded"]

corr_df = df[corr_cols].corr(numeric_only=True)

print("Correlation matrix (numeric features + encoded target):")
display(corr_df.round(3))

plt.figure(figsize=(8, 6))
sns.heatmap(corr_df, annot=True, fmt=".2f", cmap="RdBu_r", center=0, square=True)
plt.title("Correlation heatmap")
plt.tight_layout()
plt.show()

# Extract target correlation and sort it
target_corr = corr_df["Loan_Status_encoded"].drop("Loan_Status_encoded")
target_corr = target_corr.iloc[(-target_corr.abs()).argsort()]
print("Strongest linear relationships with Loan_Status (encoded):")
print(target_corr)
```



## 8.Data Standardization

Data standardization is the process of transforming numerical features so that they have a **mean of 0 and a standard deviation of 1**.

This step is important because many machine learning algorithms (such as Logistic Regression, SVM, and KNN) are sensitive to the scale of the data.

### Why Standardization is Important

Before standardization:

- Features like **ApplicantIncome** have very large values
- Features like **Credit\_History** have small values (0 or 1)

This difference in scale can:

- Bias the model toward larger-value features
- Reduce model performance

### 8.1 Standardization Formula

The standardization is performed using the following formula:

---

$$z = \frac{x - \mu}{\sigma}$$

**z**: z-score (standardized score or value)

**x**: data value

**$\mu$** : mean

**$\sigma$** : standard deviation

Where:

- $(x)$  = original value
- $(\mu)$  = mean of the feature
- $(\sigma)$  = standard deviation

### Process Applied

- Standardization was applied only to **numerical features**:
  - ApplicantIncome
  - CoapplicantIncome
  - LoanAmount
  - Loan\_Amount\_Term
- Categorical features were **not standardized** because they are already encoded values

## 8.2 Results After Standardization

|     | Gender_enc | Married_enc | Dependents_enc | Education_enc | Self_Employed_enc | Property_Area_enc | Credit_History_enc | ApplicantIncome | CoapplicantIncome | LoanAmount | Loan_Amount_Term |
|-----|------------|-------------|----------------|---------------|-------------------|-------------------|--------------------|-----------------|-------------------|------------|------------------|
| 154 | 1          | 0           | 0              | 0             | 0                 | 2                 | 1                  | -0.477068       | -0.867409         | -1.710475  | 0.280691         |
| 239 | 1          | 1           | 1              | 0             | 0                 | 1                 | 1                  | -0.443901       | -0.867409         | -0.715989  | 0.280691         |
| 448 | 1          | 1           | 2              | 0             | 0                 | 0                 | 0                  | -0.430308       | 0.321718          | 0.451451   | 0.280691         |
| 471 | 1          | 1           | 1              | 1             | 0                 | 0                 | 0                  | -0.803838       | 0.175685          | -0.348462  | -2.486768        |
| 273 | 1          | 1           | 0              | 0             | 0                 | 1                 | 1                  | -0.821781       | 0.678456          | 0.451451   | 0.280691         |
| 71  | 1          | 1           | 2              | 1             | 1                 | 1                 | 1                  | -1.226846       | 0.436458          | -0.694370  | 0.280691         |

After standardization:

- All numerical variables are centered around **0**
- Variance becomes consistent across features
- Data becomes more suitable for machine learning algorithms

## Conclusion of Data Standardization

- Standardization improves model performance and stability
- It ensures that all features contribute equally to the model
- It is an essential preprocessing step before training most ML models

## Code Section :

```
X_cols = [c for c in encoded_cat_names if c != "Loan_Status_encoded"] + numeric_cols
X = df[X_cols].copy()
y = df["Loan_Status_encoded"].copy()

print(f"Feature matrix X shape: {X.shape}")
print(f"Target y shape: {y.shape}")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

[ ]
... Feature matrix X shape: (614, 11)
Target y shape: (614,)
```

```
scaler = StandardScaler()
X_train_sc = X_train.copy()
X_test_sc = X_test.copy()
X_train_sc[numeric_cols] = scaler.fit_transform(X_train[numeric_cols])
X_test_sc[numeric_cols] = scaler.transform(X_test[numeric_cols])

display(X_train_sc.head(6))
```

```
y_train
[24]
... 154 1
    239 1
    448 0
    471 0
    273 1
    ..
    222 1
    349 1
    26 1
    96 1
    354 1
    Name: Loan_Status_encoded, Length: 491, dtype: int64
```